

Security Audit

Stoneii (Defi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	27
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Stoneii team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Stoneii Smart is a smart-contract suite designed to support compliant on-chain fundraising and private-equity style token offerings on EVM networks. The system includes infrastructure for token presales, private equity issuance, fee management, and admin recovery, alongside a KYC Soulbound token to enforce identity and eligibility requirements. By combining routers, factories, and compliance modules, the project aims to provide a structured, regulation-aware framework for launching and managing tokenized investment products in a secure and transparent way.

Project Name: Stoneii

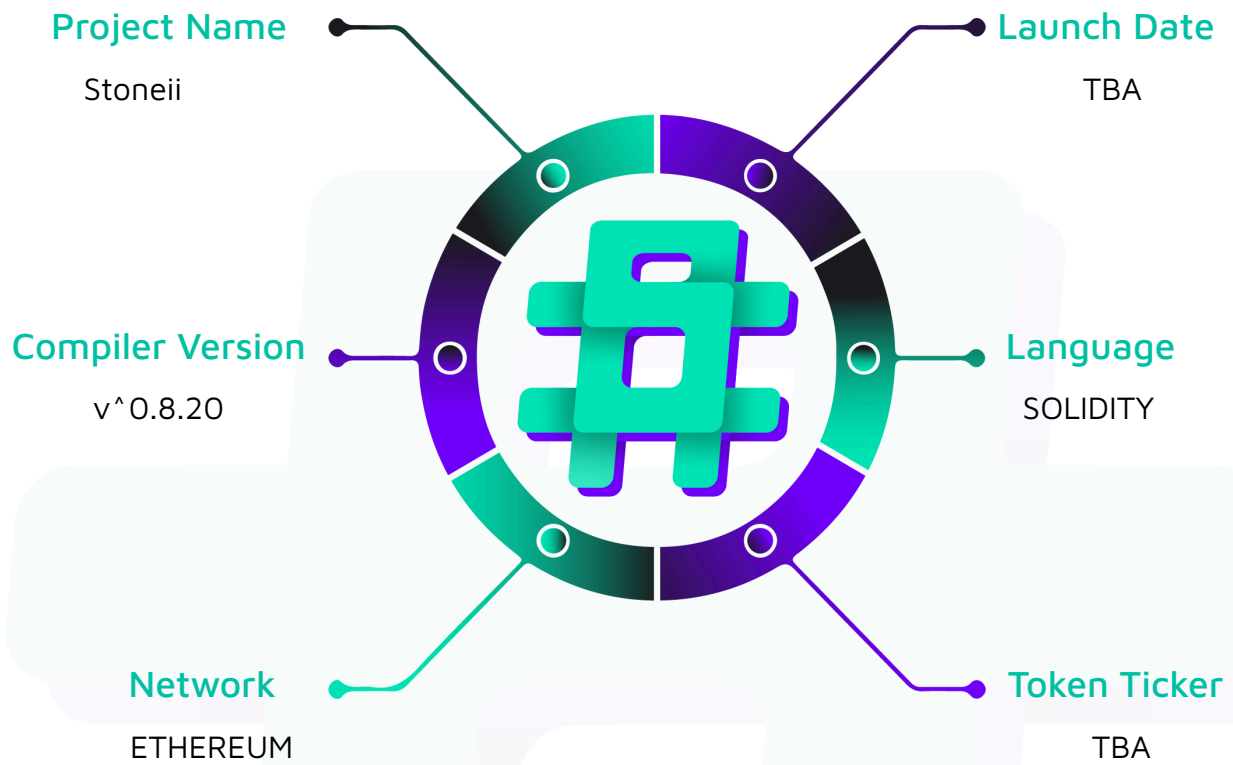
Project Type: Defi

Compiler Version: ^0.8.20

Website: <https://www.stoneii.com/>

Logo:



Visualised Context:

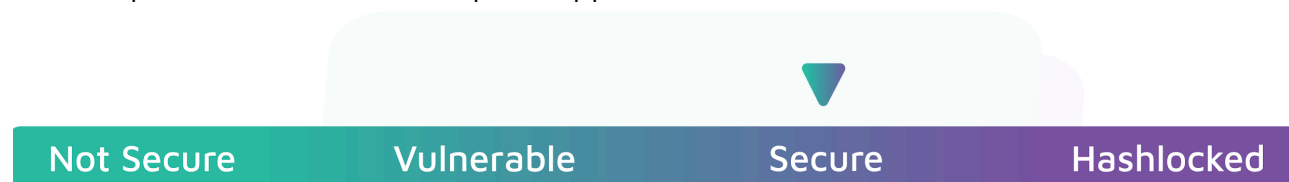
Audit Scope

We at Hashlock audited the solidity code within the Stoneii project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Stoneii Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	February, 2026
Contract 1	KYCSoulBound.sol
Contract 2	PresaleFactory.sol
Contract 3	PresaleRouterContract.so
Contract 4	AdminRescue.sol
Contract 5	Compliance.sol
Contract 6	FeeManager.sol
Contract 7	PrivateEquityRouter.sol
Contract 8	PrivateEquityToken.sol
Audited GitHub Commit Hash	9791c15a7acafc8f140d97a9d6c4465ff9971ae4
Fix Review GitHub Commit Hash	4cfadb3a1c11044a788b8bb83b5538c28c2f85e1

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

5 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
KYCSoulBound.sol - Allows users to: - Mint a KYC SBT for a user - Check SBT ownership status - Allows admins to: - Revoke a user's SBT - Add or remove admin roles	Contract achieves this functionality.
PresaleRouterContract.sol - Allows users to: - Buy presale tokens with USDC - Claim tokens after success - Claim refunds after failure - Allows admins to: - Finalize presale success or failure - Create the real token via router	Contract achieves this functionality.
PresaleFactory.sol - Allows users to: - View deployed presale contracts - Allows admins to: - Deploy new presale contracts - Set the router address - Configure admin fee recipient	Contract achieves this functionality.
AdminRescue.sol - Allows users to: - None	Contract achieves this functionality.

- Allows admins to: - Rescue ETH from the contract - Rescue ERC20 tokens from the contract	
Compliance.sol - Allows users to: - Check SBT ownership status - Query transfer/mint/burn allowance - Allows admins to: - Update router address - Set whitelist entries - Set SBT contract address	Contract achieves this functionality.
FeeManager.sol - Allows users to: - None - Allows admins to: - None	Contract achieves this functionality.
PrivateEquityRouter.sol - Allows users to: - Buy tokens with USDC - Redeem tokens for USDC - Trigger annual fee payments - Allows admins to: - Create and configure new tokens - Update price, cap, minting, redeem status - Manage whitelist and SBT address - Fund redemption liquidity	Contract achieves this functionality.
PrivateEquityToken.sol - Allows users to: - Transfer tokens (with router checks) - TransferFrom tokens (with router checks)	Contract achieves this functionality.

- Allows admins to: - Mint tokens - Burn tokens - Update token cap	
---	--



Code Quality

This audit scope involves the smart contracts of the Stoneii project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Stoneii project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] KYCSoulBound#mintSBT - Unrestricted KYC SBT Minting

Description

The `mintSBT` function is external and has no role check, so any address can mint a KYC SBT for itself or others. This bypasses the intended KYC gate because downstream flows only check SBT balance, not the issuer. An attacker can self-issue the credential and access presale/private-equity minting that should be restricted.

Vulnerability Details

ADMIN_ROLE exists and is enforced for revocation, but minting omits any access control. Because the SBT is non-transferable, minting is the only path to obtain it. Compliance checks in `src/RouterContracts/PrivateEquity/Compliance.sol` and presale buy in `src/RouterContracts/PresaleToken/PresaleRouterContract.sol` only verify SBT ownership. Therefore, a forged SBT satisfies all current gating checks.

```
// src/libraries/KYCSoulBound.sol
function mintSBT(address user) external {
    require(balanceOf(user) == 0, "User already has SBT");
    uint256 tokenId = uint256(uint160(user));
    _mint(user, tokenId);
}
```

Proof of Concept

A Foundry test below demonstrates that a non-admin address can mint an SBT.

```
function testUnrestrictedSBTMint() public {
    // --- Setup and Preconditions ---
    KYCSoulBound sbt = new KYCSoulBound();
    address attacker = address(0xBEEF);
    uint256 beforeBal = sbt.balanceOf(attacker);
```

```
// --- Exploit Execution ---
vm.prank(attacker);
sbt.mintSBT(attacker);

// --- Post-condition Check ---
uint256 afterBal = sbt.balanceOf(attacker);
assertGt(afterBal, beforeBal); // PoC succeeds if attacker minted an SBT
}
```

Impact

Any user can self-issue the KYC SBT and bypass gating for presales and private-equity token minting, undermining compliance controls and enabling unauthorized participation in restricted markets, which can lead to regulatory risk and improper token distribution.

Recommendation

Restrict minting with `onlyRole(ADMIN_ROLE)` or a dedicated `MINTER_ROLE`. If open minting is desired for testing only, gate it behind a test-only flag or deploy-time configuration. Consider emitting explicit mint events for auditability.

```
- function mintSBT(address user) external {
+ function mintSBT(address user) external onlyRole(ADMIN_ROLE) {
```

Status

Resolved

[H-02] PrivateEquityRouter#constructor - AdminRescue Ownership Locked to Router

Description

PrivateEquityRouter sets the AdminRescue owner to the router itself in the constructor. AdminRescue's rescue functions require `msg.sender` to be that owner, and the router exposes no forwarder. As a result, the protocol owner cannot rescue ETH or ERC20s accidentally sent to the router.

Vulnerability Details

AdminRescue restricts rescue operations via `onlyRouterisOwner`, which compares `msg.sender` to `_owner`. The router constructor sets `_owner` to `address(this)`, not the `Ownable` owner. No external or internal router method performs a self-call to `rescueETH` or `rescueERC20`.

Therefore, no EOA can ever satisfy the `onlyRouterisOwner` check.

```
// src/RouterContracts/PrivateEquity/PrivateEquityRouter.sol
_setAdminRescueOwner(address(this));

// src/RouterContracts/PrivateEquity/AdminRescue.sol
modifier onlyRouterisOwner() {
    require(msg.sender == _owner, "Only Owner can perform this action");
    _;
}
```

Proof of Concept

A Foundry test below shows that the protocol owner cannot rescue ERC20 funds from the router.

```
function testAdminRescueLocked() public {
    // --- Setup and Preconditions ---
    uint256 fundAmount = 100e6;
    vm.startPrank(owner);
    usdc.approve(address(router), fundAmount);
    router.adminFundUSDC(fundAmount);
}
```

```

vm.stopPrank();

uint256 routerBalBefore = usdc.balanceOf(address(router));

// --- Exploit Execution ---
bool success;
vm.prank(owner);
try router.rescueERC20(address(usdc)) {
    success = true;
} catch {
    success = false;
}

// --- Post-condition Check ---
uint256 routerBalAfter = usdc.balanceOf(address(router));
assertTrue(!success); // PoC succeeds if owner cannot rescue ERC20
assertEq(routerBalAfter, routerBalBefore); // PoC succeeds if funds stay locked
}

```

Impact

Any ETH or ERC20 accidentally transferred to the router becomes permanently stuck, preventing administrative recovery of protocol funds and increasing operational risk during incidents or migrations.

Recommendation

Initialize the AdminRescue owner to the `Ownable` owner rather than the router address. If ownership can change, add a hook to keep the AdminRescue owner in sync with `transferOwnership`. Alternatively, redesign AdminRescue to inherit `Ownable` and use `onlyOwner` for rescue functions.

```

- _setAdminRescueOwner(address(this));
+ _setAdminRescueOwner(owner());

```

Status

Resolved

[H-03] Presale#constructor - Raised USDC Is Permanently Locked After Successful Settlement

Description

`Presale.buy` transfers USDC from users into the `Presale` contract and increments `totalRaised`. If the presale is finalized as successful via `settleSuccess`, there is no mechanism to transfer the raised USDC to the project/treasury. The only USDC outflow is `claimRefund`, which is gated to the `Failed` status and cannot be used after success. As a result, all USDC contributed during a successful presale can remain permanently locked in the contract.

Vulnerability Details

In `buy`, USDC is pulled into the contract using `usdc.transferFrom(...)`. The contract tracks deposits via `contributed` and `totalRaised`, but never forwards the collected USDC to any recipient. In `settleSuccess`, the owner sets `status = PresaleStatus.Successful` and creates the real token via `router.createToken(...)` without transferring USDC. The only function that transfers USDC out is `claimRefund`, which requires `status == PresaleStatus.Failed`. There is no owner-only withdrawal/sweep function and no code path that moves USDC out when `status == Successful`. Therefore, once the presale succeeds, the raised USDC balance is stuck at `address(this)` indefinitely.

```
// src/RouterContracts/PresaleToken/PresaleRouterContract.sol
function buy(uint256 usdcAmount) external {
    usdc.transferFrom(msg.sender, address(this), usdcAmount); // funds enter Presale
}

function settleSuccess(...) external onlyOwner {
    status = PresaleStatus.Successful;
    realTokenId = router.createToken(...);
    // no USDC transfer to owner/treasury
}

function claimRefund() external {
    require(status == PresaleStatus.Failed, "Presale not failed"); // blocks refunds on
    success
}
```

```
usdc.transfer(msg.sender, amount); // only outflow path
}
```

Impact

If the presale is finalized as `Successful`, up to `totalRaised` USDC (potentially the entire amount contributed by users) becomes unrecoverable because the contract provides no callable path to transfer the collected USDC to the project treasury, breaking the core presale settlement flow and leaving the full USDC balance permanently stranded in the `Presale` contract.

Recommendation

Add an owner-only withdrawal function to recover collected USDC. Use `SafeERC20.safeTransfer` for safer ERC20 transfers.

Consider restricting withdrawal to only the `Successful` state to preserve refund guarantees in the `Failed` path.

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

contract Presale is Ownable {
+   using SafeERC20 for IERC20;

+   function withdrawUSDC(address to) external onlyOwner {
+       uint256 bal = usdc.balanceOf(address(this));
+       usdc.safeTransfer(to, bal);
+   }
}
```

Status

Resolved

Medium

[M-01] PrivateEquityRouter#buyTokens/redeem - No Slippage Bounds on Price

Description

Both `buyTokens` and `redeem` use the live `adminPriceUSDC` without user-provided minimums or deadlines. Users cannot bound the USDC they pay or receive if the price changes between signing and execution.

Vulnerability Details

The router reads `cfg.adminPriceUSDC` at execution time and applies it directly to cost and redemption value. There is no caller-specified price bound or `deadline` to protect against stale or adverse price updates. A privileged owner or fast actor can update the price in the same block, changing settlement terms. This creates avoidable user loss and a poor UX for buy and redeem flows.

```
// src/RouterContracts/PrivateEquity/PrivateEquityRouter.sol
uint256 currentPriceUSDC = cfg.adminPriceUSDC;
uint256 costUSDC = (currentPriceUSDC * amount) / 1e18;
```

Impact

Users can be forced to overpay or under-receive USDC when prices change between transaction submission and execution, leading to unpredictable outcomes and reduced trust in the protocol's pricing mechanics.

Recommendation

Add user-specified bounds such as `maxCostUSDC`, `minRedeemUSDC`, and a `deadline` parameter. Reject transactions if the live price violates those bounds. Document the pricing update process and expected user protections.

Status

Resolved

[M-02] PrivateEquityRouter#redeem - Price Update Front-Run Arbitrage

Description

The redeem price is read directly from `cfg.adminPriceUSDC` at execution time. The owner can update this price at any time, and there is no slippage or deadline protection. An attacker who anticipates a price increase can buy before the update and redeem immediately after for profit.

Vulnerability Details

Price is owner-controlled via `updatePrice`, and redemption uses the current stored value without user-specified constraints. If the router has USDC liquidity, any holder can redeem at the updated price immediately after the change. This enables predictable arbitrage around price updates and can transfer value from the protocol's USDC pool. The risk is amplified when updates are announced or observable in the mempool.

```
// src/RouterContracts/PrivateEquity/PrivateEquityRouter.sol
uint256 currentPriceUSDC = cfg.adminPriceUSDC;
uint256 redeemValue = (currentPriceUSDC * amount) / 1e18;
```

Impact

Predictable price updates can be arbitrated by privileged or fast actors, potentially draining USDC liquidity from the router and undermining fair pricing for ordinary users during redemption windows.

Recommendation

Consider enforcing a time delay or oracle-based TWAP for `updatePrice` to reduce front-running risk. Document the intended price update process to set user expectations.

Status

Acknowledged

Low

[L-01] AdminRescue#rescueERC20 - Unsafe ERC20 Rescue Transfer

Description

`rescueERC20` relies on `IERC20.transfer` returning `true` and does not use `SafeERC20`. Non-standard ERC20s that return no value or revert can cause the rescue to fail, leaving funds stuck.

Recommendation

Use `SafeERC20` and `safeTransfer` to support non-standard tokens. If only fully compliant tokens are supported, document this constraint explicitly. Consider emitting rescue events for operational visibility.

Status

Resolved

[L-02] PrivateEquityRouter#buyTokens - Purchases Do Not Retain Redemption Liquidity

Description

`buyTokens` transfers the entire USDC payment to `feeRecipient`, leaving the router with no redemption liquidity. Redemptions then rely on manual `adminFundUSDC`, which creates operational risk if funds are not topped up.

Recommendation

If the protocol expects automatic redemption liquidity, retain the purchase cost in the router and forward only fees. Alternatively, document that redemptions require separate treasury funding and monitor liquidity levels. Add safeguards to block redemptions when funding is insufficient to avoid partial failures.

Status

Resolved

QA

[Q-01] KYCSoulBound#addAdmin/removeAdmin - Redundant Admin Role Helpers

Description

`addAdmin` and `removeAdmin` wrap AccessControl's `grantRole`/`revokeRole` without extra logic. This duplicates functionality already available to `DEFAULT\_ADMIN\_ROLE` holders.

Recommendation

Keep these helpers if they improve clarity for integrators. If minimizing surface area is preferred, remove them and rely on AccessControl's built-in functions. Document the intended role-management pattern in contract comments.

Status

Acknowledged

[Q-02] Contract - Missing Zero-Address Validation in Core Constructors

Description

Multiple constructors accept critical addresses with no zero-address validation, risking misconfiguration.

Affected locations: ``src/RouterContracts/PresaleToken/PresaleRouterContract.sol``,
``src/RouterContracts/PresaleToken/PresaleFactory.sol``,
``src/RouterContracts/PrivateEquity/Compliance.sol``, and
``src/RouterContracts/PrivateEquity/PrivateEquityRouter.sol``.
``settleSuccess`` also lacks a zero-address guard for inputs in
``src/RouterContracts/PresaleToken/PresaleRouterContract.sol``.

Recommendation

Add ``require(addr != address(0))`` checks for critical dependency addresses at construction and settlement. If any zero-address values are meant to disable features, document those cases explicitly. Consider using custom errors for clearer misconfiguration diagnostics.

Status

Resolved

[Q-03] PresaleFactory - Unused adminFeeRecipient Variable

Description

The `'adminFeeRecipient'` state variable is stored but never referenced. This indicates dead code or a missing fee-routing feature in the presale factory.

Recommendation

Remove `'adminFeeRecipient'` if it is not needed. If an admin fee flow is intended, wire this value into presale settlement or fee distribution logic. Add tests that assert the expected fee behavior.

Status

Resolved

[Q-04] FeeManager#setLastFeeTimestamp/setHasSubscribed - Internal Naming Deviates from Convention

Description

Internal setter names `setLastFeeTimestamp` and `setHasSubscribed` do not follow the project's preferred naming guidance. This is a consistency/readability issue rather than a functional defect.

Recommendation

Rename the setters to match the project's naming conventions. If no convention is enforced, add a brief note in the coding guidelines. Ensure refactors are reflected in tests and external documentation.

Status

Acknowledged

[Q-05] PrivateEquityRouter#mintFromPresale - Presale Mint Lacks Explicit Cap Check (Redundant)

Description

`mintFromPresale` does not perform an explicit cap check before minting. However, `PrivateEquityToken.mint` already enforces the cap, making this a redundancy/clarity concern.

Recommendation

Optionally add an explicit cap check in `mintFromPresale` for readability. If redundancy is undesirable, remove the QA comment and rely on the token-level guard. Document that cap enforcement lives in the token contract.

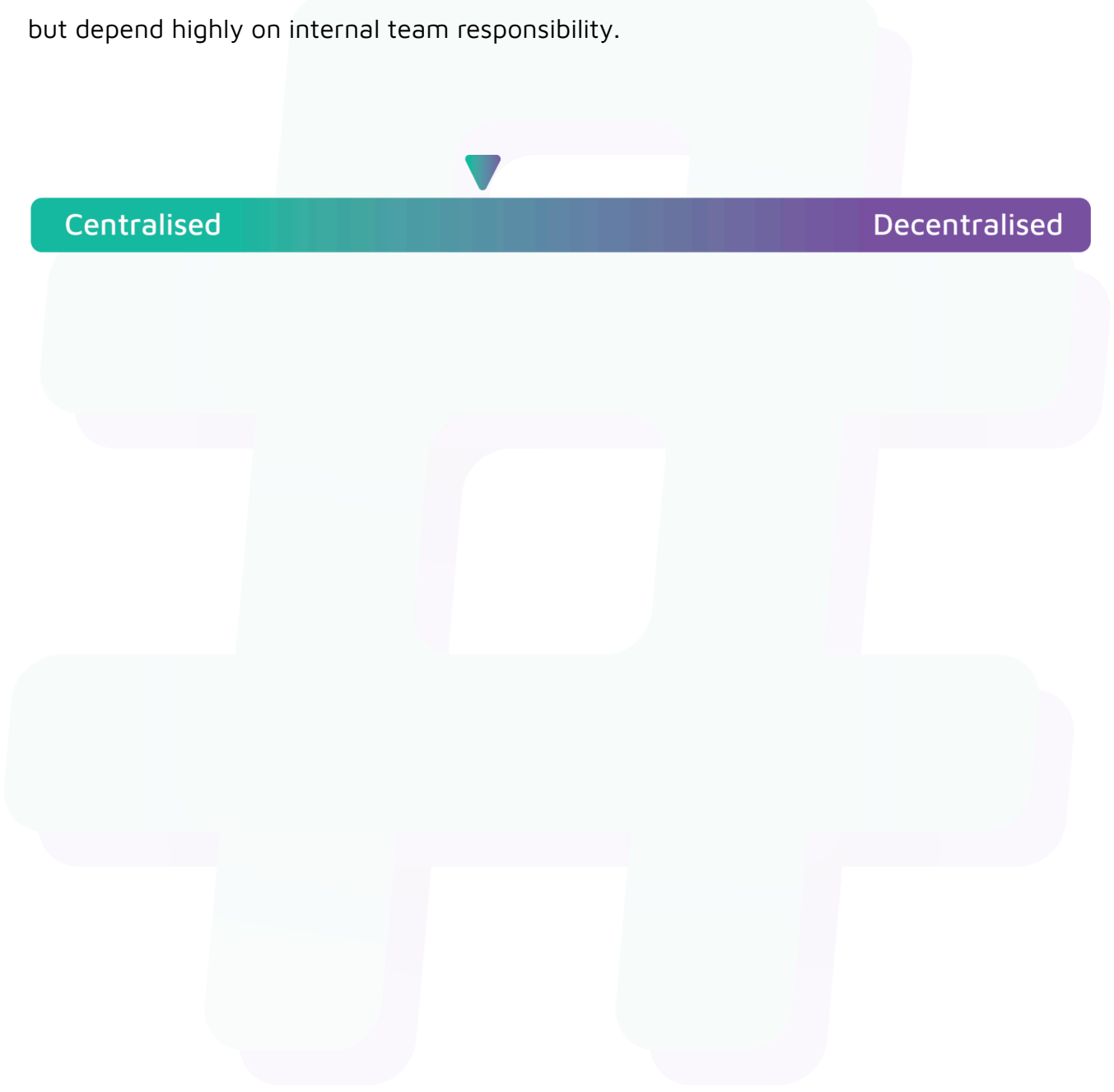
Status

Acknowledged

Centralisation

The Stoneii project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Stoneii project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.